Free offer Share

Monday 11:22 AM

Pasted markdown.md
File

DC_Complexlab Racoon POD 05 Purple

Headquarter (HQ) and CoomandPost (CP) should be implement

1. Topology

Teamwork: two participants per POD

Each
POD is configured by two participants. One participant takes primary

Show more

Yes. For **Classroom 1 – Raccoon, POD 05 – Purple**, the target is one AD forest/domain spanning two sites: **HQ** and **CP**. The topology specifies 14 POD devices, with four DCs total —two at each site. Pasted markdown

POD 05 Purple — implementation target

1. HQ site

Device	IP	Role
LR01	`10.101.50.254/24`	Router, firewall, WireGuard
HV01	`10.101.50.10/24`	Proxmox
HQ-DC01	`10.101.50.11/24`	AD DS + DNS, first DC
HQ-DC02	`10.101.50.12/24`	AD DS + DNS, second DC
HQ-FS01	`10.101.50.20/24`	File/SMB server

Device	IP	Role
HQ-CA	`10.101.50.30/24`	Subordinate CA
HQ-SRV	`10.101.50.40/24`	Linux; later Docker/checkmk/Graylog
HQ-APC	`10.101.50.100/24`	Administration/RDP

The HQ LAN is `10.101.50.0/24`, with LR01 as gateway. Pasted markdown The individual HQ devices and their functions are specified here. Pasted markdown

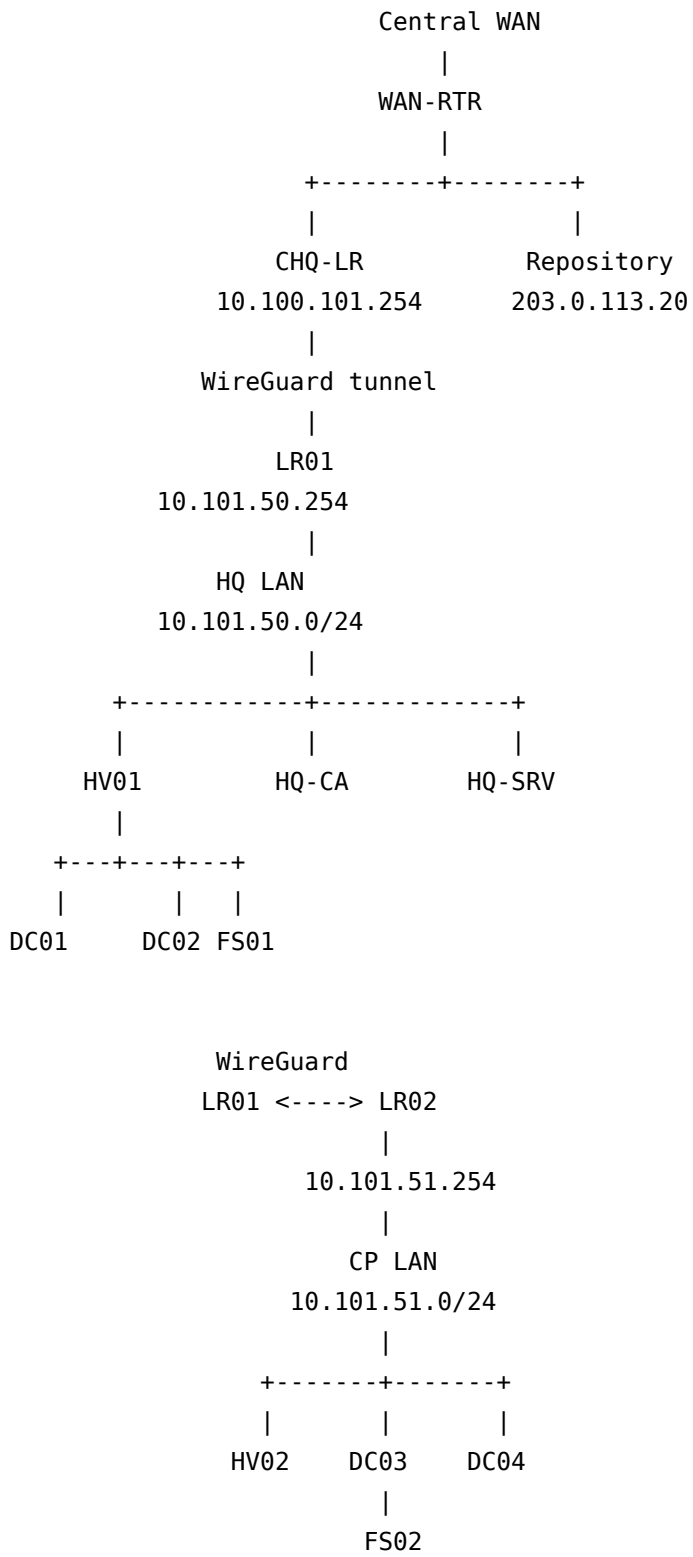
2. CP site

Device	IP	Role
LR02	`10.101.51.254/24`	Router, firewall, WireGuard
HV02	`10.101.51.10/24`	Proxmox
CP-DC03	`10.101.51.11/24`	AD DS + DNS
CP-DC04	`10.101.51.12/24`	AD DS + DNS
CP-FS02	`10.101.51.20/24`	File/SMB server
CP-APC	`10.101.51.100/24`	Administration/RDP

CP uses `10.101.51.0/24` and LR02 is its gateway. Pasted markdown The CP device assignments are defined here. Pasted markdown

Recommended implementation order

I would build the POD in this order:



The important point is that **HQ and CP are not two separate AD domains**. They are two AD sites/subnets in the **same domain and forest**:

```
Forest: purple.raccoon.lab
Domain: purple.raccoon.lab

AD Site: HQ
    10.101.50.0/24
    HQ-DC01
    HQ-DC02

AD Site: CP
    10.101.51.0/24
    CP-DC03
    CP-DC04
```

The source explicitly says there is one `purple.raccoon.lab` domain, four DCs, and that the HQ/CP LAN subnets should be associated with their respective AD sites.

Pasted markdown

Phase 1 — Build the network

HQ LR01

Configure:

```
LAN:
10.101.50.254/24

WAN:
203.0.113.162/30
Gateway:
203.0.113.161
```

The WAN `/30` is:

```
203.0.113.160/30

WAN-RTR = 203.0.113.161
LR01    = 203.0.113.162
```

Pasted markdown

CP LR02

```
LAN:
10.101.51.254/24

WAN:
203.0.113.166/30
Gateway:
203.0.113.165
```

The CP transit network is:

```
203.0.113.164/30

WAN-RTR = 203.0.113.165
LR02    = 203.0.113.166
```

Pasted markdown

Phase 2 — WireGuard

There are **two tunnels**:

CentralHQ ↔ HQ

```
10.250.101.20/30

CHQ-LR = 10.250.101.21
LR01   = 10.250.101.22
```

HQ ↔ CP

```
10.251.101.20/30

LR01 = 10.251.101.21
LR02 = 10.251.101.22
```

Pasted markdown

Routing should therefore be:

CHQ-LR

```
10.101.50.0/24 -> LR01
10.101.51.0/24 -> LR01
```

LR01

```
10.100.101.0/24 -> CHQ-LR
10.101.51.0/24  -> LR02
```

LR02

```
10.101.50.0/24  -> LR01
10.100.101.0/24 -> LR01
```

This is the topology's prescribed routing model. Pasted markdown

Important: no NAT inside WireGuard. IP forwarding must be enabled on the Linux routers.

Pasted markdown

Phase 3 — Proxmox

HV01

```
10.101.50.10/24
gateway 10.101.50.254
```

Create three VMs:

```
HQ-DC01   10.101.50.11
HQ-DC02   10.101.50.12
HQ-FS01   10.101.50.20
```

HV02

```
10.101.51.10/24
gateway 10.101.51.254
```

Create:

```
CP-DC03   10.101.51.11
CP-DC04   10.101.51.12
CP-FS02   10.101.51.20
```

The lab specifically requires two DCs and one file server on each Proxmox host.

Pasted markdown

Phase 4 — Active Directory

Create the forest/domain on **HQ-DC01**:

```
Domain:
purple.raccoon.lab

Forest:
purple.raccoon.lab
```

Then promote:

```
HQ-DC01  → first DC
HQ-DC02  → additional DC
CP-DC03  → additional DC
CP-DC04  → additional DC
```

DNS should be AD-integrated.

The resulting structure should conceptually be:

```
purple.raccoon.lab
│
├── HQ
│   ├── HQ-DC01
│   └── HQ-DC02
│
└── CP
    ├── CP-DC03
    └── CP-DC04
```

But **HQ and CP should be AD Sites**, not child domains.

Phase 5 — AD Sites and Services

This is particularly important.

Create:

```
Sites
├── HQ
└── CP
```

Then create the corresponding subnets:

```
10.101.50.0/24 → HQ
10.101.51.0/24 → CP
```

Place the DCs accordingly:

```
HQ
 ├── HQ-DC01
 └── HQ-DC02

CP
 ├── CP-DC03
 └── CP-DC04
```

This ensures clients at HQ preferentially use HQ DCs and CP clients preferentially use CP DCs.

Phase 6 — DNS

For normal domain DNS:

HQ clients

Prefer:

```
10.101.50.11
10.101.50.12
```

CP clients

Prefer:

```
10.101.51.11
10.101.51.12
```

The topology additionally requires a conditional forwarder for the central domains:

```
chq.raccoon.lab → 10.100.101.254
wan.raccoon.lab → 10.100.101.254
```

and explicitly says **not to forward to the other classroom**. Pasted markdown

Phase 7 — File servers

Create/configure:

```
HQ-FS01
10.101.50.20

CP-FS02
10.101.51.20
```

They should be domain members of:

```
purple.raccoon.lab
```

and provide the required SMB/file shares.

Phase 8 — HQ-CA

HQ-CA:

```
hq-ca01.purple.raccoon.lab
10.101.50.30
```

It is **not the Root CA**.

The hierarchy is:

```
CHQ-CA
   Root CA
      |
      └── HQ-CA
            SubCA
            |
            +── HQ certificates
            └── CP certificates
```

The source specifies that the classroom Root CA signs the seven HQ SubCAs and that HQ-CA issues certificates for both HQ and CP. Pasted markdown +1

Phase 9 — APCs

HQ-APC:

```
LAN: 10.101.50.100/24
Gateway for LAN routes: 10.101.50.254
```

CP-APC:

```
LAN: 10.101.51.100/24
Gateway for LAN routes: 10.101.51.254
```

The APCs also have an **extNet interface**, which is where RDP access occurs. The trainer supplies the actual extNet/RDP addresses. Pasted markdown

Do **not** configure:

```
ICS
RRAS
IP forwarding
LAN bridge
```

on the APCs.

Only the extNet interface should have the default gateway. Pasted markdown

Phase 10 — Connectivity verification

Before doing higher-level AD/PKI work, verify this sequence.

HQ

```
HQ-DC01 → 10.101.50.254
HQ-DC02 → 10.101.50.254
HQ-FS01 → 10.101.50.254
HQ-CA  → 10.101.50.254
HQ-SRV → 10.101.50.254
```

CP

```
CP-DC03 → 10.101.51.254
CP-DC04 → 10.101.51.254
CP-FS02 → 10.101.51.254
```

Inter-site

From HQ:

```
ping 10.101.51.11
ping 10.101.51.12
ping 10.101.51.20
```

From CP:

```
ping 10.101.50.11
ping 10.101.50.12
ping 10.101.50.20
```

CentralHQ

From HQ:

```
ping 10.100.101.254
```

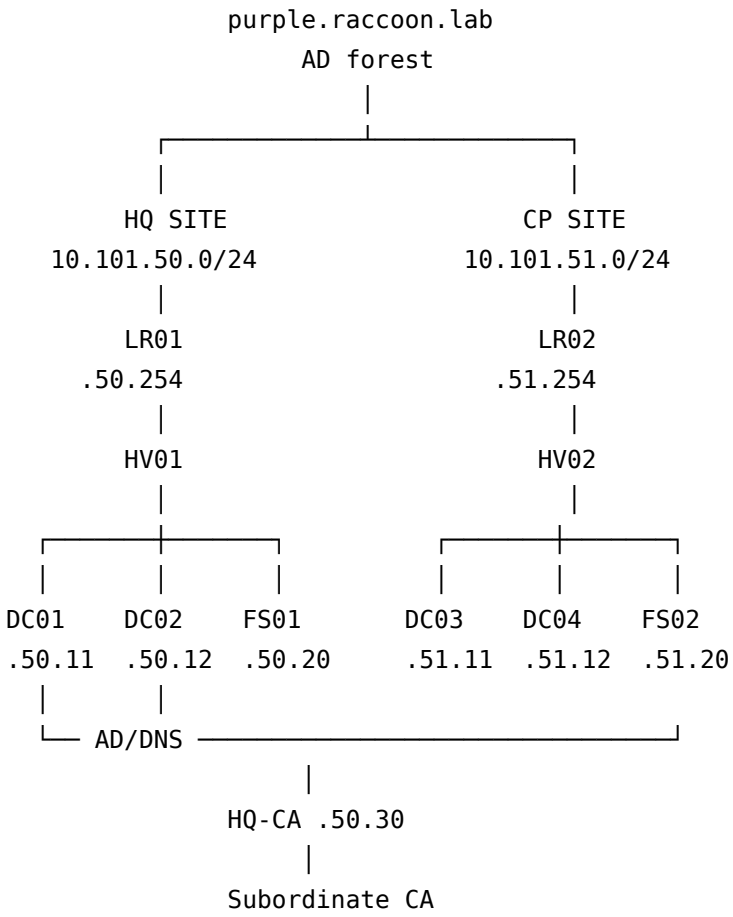
and, where permitted:

```
ping 10.100.101.20
ping 10.100.101.30
```

The actual service protocols should also be tested; ICMP alone isn't enough.

Final target

When complete, your **POD 05 Purple** should look like this:



The critical architectural rule is: **one domain/forest, two AD sites, four DCs, two routed private LANs, and WireGuard for all private-site communication.** The source explicitly prohibits using direct WAN routing/NAT/bridging as a substitute for the WireGuard topology. Pasted markdown

If you're going to **actually build this lab now**, I can take this topology and give you the **exact step-by-step implementation commands**, starting with **LR01/LR02 → Proxmox → DC01 → AD Sites → DC02/DC03/DC04**, including the exact Linux and PowerShell

commands for POD 05.

Sources

Let's work through **Exercise 14 – PKI** for your `purple.raccoon.lab` POD, using **HQ-CA01** as the subordinate CA and **HQ-SRV (** `10.101.50.40` **)** as the HTTP publication server.

We'll do this in checkpoints. **Don't install AD CS until the DNS/publication prerequisites are working**, because the AIA/CDP configuration later depends on them.

Phase 1 — Prepare HQ-CA01

1. Verify HQ-CA01 network configuration

On **HQ-CA01**, run:

PowerShell

```
hostname
ipconfig
ipconfig /all
```

Expected topology:

```
Hostname: HQ-CA01
IP:       10.101.50.30
Mask:     255.255.255.0
Gateway:  10.101.50.254
DNS:      10.101.50.11 / 10.101.50.12
```

For a domain member, don't configure public DNS directly on the NIC.

Test:

PowerShell

```
Test-Connection HQ-DC01 -Count 2
Test-Connection HQ-DC02 -Count 2
Test-Connection 10.101.50.254 -Count 2
```

Then:

PowerShell

```
Resolve-DnsName purple.raccoon.lab
Resolve-DnsName HQ-DC01.purple.raccoon.lab
```

2. Verify domain membership

On HQ-CA01:

PowerShell

```
Get-CimInstance Win32_ComputerSystem |
    Select-Object Name,Domain,PartOfDomain
```

Expected:

```
Name          : HQ-CA01
Domain        : purple.raccoon.lab
PartOfDomain  : True
```

Also:

PowerShell

```
whoami
```

You should be operating with an account that has the permissions required to install/configure an Enterprise subordinate CA.

3. Verify AD replication

From HQ-CA01:

PowerShell

```
repadmin /replsummary
```

You want no persistent replication failures.

Also:

PowerShell

```
nltest /dsgetdc:purple.raccoon.lab
```

Expected to return one of your domain controllers.

And:

PowerShell

```
nltest /dclist:purple.raccoon.lab
```

You should eventually have:

```
HQ-DC01
HQ-DC02
CP-DC03
CP-DC04
```

4. Verify time

On HQ-CA01:

PowerShell

```
w32tm /query /status
w32tm /query /source
```

Then:

PowerShell

```
w32tm /resync
```

For AD CS, make sure the domain time hierarchy is healthy rather than manually configuring an unrelated public NTP source on the CA.

Phase 2 — Create `pki.local`

Your exercise specifically requires an **AD-integrated forward lookup zone**:

```
pki.local
```

and an A record at the zone root pointing to:

```
10.101.50.40
```

which is HQ-SRV.

5. Create the DNS zone

On **HQ-DC01**, open:

```
dnsmgmt.msc
```

Navigate:

```
DNS
 └─ HQ-DC01
     └─ Forward Lookup Zones
```

Right-click:

```
Forward Lookup Zones
 → New Zone...
```

Wizard:

Zone Type

Choose:

```
Primary zone
```

Because this is an AD-integrated DNS server, you should see the option:

```
Store the zone in Active Directory
```

Select it.

Replication Scope

Choose:

```
To all DNS servers running on domain controllers in this forest
```

This makes `pki.local` available throughout your AD DNS infrastructure.

Zone Name

Enter:

```
pki.local
```

Dynamic Updates

For this lab, use:

```
Allow only secure dynamic updates
```

if offered/appropriate for your AD-integrated zone.

Finish.

6. Create the root A record

Open:

```
Forward Lookup Zones
 └─ pki.local
```

Right-click:

```
pki.local
 → New Host (A or AAAA)...
```

Leave **Name** completely empty.

This is important.

Set:

```
IP address: 10.101.50.40
```

Do **not** create:

```
www.pki.local
```

The exercise explicitly wants the zone root:

```
pki.local
```

So DNS should resolve:

```
pki.local → 10.101.50.40
```

Test from HQ-CA01:

PowerShell

```
Resolve-DnsName pki.local
```

Expected:

```
Name       Type   IPAddress
----       ----   ---------
pki.local  A      10.101.50.40
```

Test from CP later as well.

7. Verify zone replication

On HQ-DC01:

PowerShell

```
Get-DnsServerZone -Name "pki.local" |
    Select ZoneName,ZoneType,IsDsIntegrated,ReplicationScope
```

You want:

```
ZoneName        : pki.local
IsDsIntegrated  : True
```

Then on another DC, for example CP-DC03:

PowerShell

```
Resolve-DnsName pki.local
```

It should return:

```
10.101.50.40
```

Phase 3 — Deploy Nginx publication service

Your publication URL will be:

```
http://pki.local/pki/
```

and the actual files will live on:

```
HQ-SRV
/opt/www/pki/
```

8. Prepare HQ-SRV

SSH to:

```
HQ-SRV
10.101.50.40
```

Create the directories:

Bash

```
sudo mkdir -p /opt/pki-web /opt/www/pki
sudo chmod 755 /opt/www
sudo chmod 755 /opt/www/pki
```

Verify:

Bash

```
ls -ld /opt/www /opt/www/pki
```

You want the directories traversable/readable by the Nginx container.

9. Create the Nginx Compose project

Bash

```
cd /opt/pki-web
```

Create `.env` :

Bash

```
sudo nano /opt/pki-web/.env
```

For your Raccoon POD:

```
REGISTRY=203.0.113.20:5000
NGINX_TAG=<trainer-provided-tag>
HQ_SRV_IP=10.101.50.40
```

The exact `NGINX_TAG` must be the tag supplied by your trainer.

Don't guess it.

10. Verify Docker's insecure registry

Because the exercise explicitly says the Repository registry is insecure, check:

Bash

```
sudo docker info | grep -A10 -i "insecure registr"
```

If you need to inspect Docker's configuration:

Bash

```
cat /etc/docker/daemon.json
```

You should have the repository registry configured according to your previous container exercise.

For example, conceptually:

JSON

```
{
  "insecure-registries": [
    "203.0.113.20:5000"
  ]
}
```

Don't overwrite an existing `daemon.json` blindly.

11. Create `compose.yaml`

Bash

```
sudo nano /opt/pki-web/compose.yaml
```

Use:

YAML

```
services:
  nginx:
    image: "${REGISTRY}/library/nginx:${NGINX_TAG}"
    restart: unless-stopped
    ports:
      - "${HQ_SRV_IP}:80:80"
    volumes:
      - /opt/www:/usr/share/nginx/html:ro
```

Save it.

12. Validate Compose

Bash

```
cd /opt/pki-web
sudo docker compose config --quiet
```

No output means the Compose syntax is valid.

Then:

Bash

```
sudo docker compose pull
```

Then:

Bash

```
sudo docker compose up -d
```

Verify:

Bash

```
sudo docker compose ps
```

You want Nginx to be:

```
Up
```

Check the port:

Bash

```
sudo ss -lntp | grep ':80'
```

13. Check that existing services weren't disturbed

Because HQ-SRV already runs Checkmk/Graylog infrastructure, verify:

Bash

```
sudo docker ps
```

You should still have the existing containers.

Also:

Bash

```
sudo ss -lntp | grep -E ':80|:9000|:1514|:12201'
```

You want HTTP `80` in addition to the existing Graylog/Checkmk-related ports.

14. Test Nginx before publishing certificates

Create a temporary test file:

Bash

```
echo "PKI publication test" | sudo tee /opt/www/pki/test.txt
```

From HQ-CA01:

PowerShell

```
Invoke-WebRequest http://pki.local/pki/test.txt
```

Expected:

```
StatusCode : 200
```

Or:

PowerShell

```
curl.exe http://pki.local/pki/test.txt
```

You should receive:

```
PKI publication test
```

Also test directly:

PowerShell

```
curl.exe http://10.101.50.40/pki/test.txt
```

Then test from **CP-APC**:

PowerShell

```
curl.exe http://pki.local/pki/test.txt
```

This test is particularly useful because it proves:

```
CP LAN
 ↓
LR02
 ↓ WireGuard
LR01
 ↓
HQ LAN
 ↓
HQ-SRV
```

works for HTTP.

Once verified:

Bash

```
sudo rm /opt/www/pki/test.txt
```

Phase 4 — Obtain the classroom Root CA files

Now we move to the actual PKI.

The CentralHQ web server for your Raccoon topology is:

```
10.100.101.20
```

So on **HQ-CA01**, create a working directory:

PowerShell

```
New-Item -ItemType Directory C:\PKI-Work -Force
Set-Location C:\PKI-Work
```

Download:

```
http://10.100.101.20/pki/RootCA.crt
http://10.100.101.20/pki/RootCA.crl
```

For example:

PowerShell

```
Invoke-WebRequest `
    -Uri "http://10.100.101.20/pki/RootCA.crt" `
    -OutFile "C:\PKI-Work\RootCA.crt"

Invoke-WebRequest `
    -Uri "http://10.100.101.20/pki/RootCA.crl" `
    -OutFile "C:\PKI-Work\RootCA.crl"
```

Verify:

PowerShell

```
Get-ChildItem C:\PKI-Work
```

15. Inspect the Root CA certificate

Run:

PowerShell

```
certutil -dump C:\PKI-Work\RootCA.crt
```

Pay attention to:

```
Issuer
Subject
NotBefore
NotAfter
Thumbprint
```

Get the fingerprint more conveniently:

PowerShell

```
Get-FileHash C:\PKI-Work\RootCA.crt -Algorithm SHA256
```

But the **certificate thumbprint/fingerprint supplied by the trainer** is the authoritative value to compare against.

Do not proceed if the Root CA certificate fingerprint does not match the trainer's value.

16. Inspect the Root CRL

PowerShell

```
certutil -dump C:\PKI-Work\RootCA.crl
```

Check:

```
Issuer
This Update
Next Update
```

The CRL should be current and issued by the classroom Root CA.

Phase 5 — Publish Root CA into AD

After the trainer has confirmed the Root CA files:

On HQ-CA01, elevated PowerShell/CMD:

cmd

```
cd C:\PKI-Work
```

Publish the Root certificate:

cmd

```
certutil -f -dspublish RootCA.crt RootCA
```

Then the CRL:

cmd

```
certutil -f -dspublish RootCA.crl
```

If the second command asks for or cannot determine the appropriate DS CDP location, **stop there** and obtain the correct container/CN from the trainer.

Don't invent an AD CDP path.

Install the root certificate locally:

cmd

```
certutil -addstore -f Root RootCA.crt
```

Verify:

PowerShell

```
Get-ChildItem Cert:\LocalMachine\Root |
    Where-Object {$_.Subject -match "Root"}
```

Or:

cmd

```
certutil -store Root
```

Phase 6 — Publish Root files through HQ-SRV

Copy the two files to:

```
/opt/www/pki/
```

For example, from HQ-CA01 using SCP:

PowerShell

```
scp C:\PKI-Work\RootCA.crt linux_admin@10.101.50.40:/tmp/
scp C:\PKI-Work\RootCA.crl linux_admin@10.101.50.40:/tmp/
```

Then on HQ-SRV:

Bash

```
sudo cp /tmp/RootCA.crt /opt/www/pki/
sudo cp /tmp/RootCA.crl /opt/www/pki/
```

Permissions:

Bash

```
sudo chmod 644 /opt/www/pki/RootCA.crt
sudo chmod 644 /opt/www/pki/RootCA.crl
```

Verify:

Bash

```
ls -l /opt/www/pki/
```

Test:

PowerShell

```
curl.exe http://pki.local/pki/RootCA.crt
curl.exe http://pki.local/pki/RootCA.crl
```

Both should download successfully.

Phase 7 — Install AD CS on HQ-CA01

Now we can install the CA role.

On HQ-CA01:

PowerShell

```
Install-WindowsFeature ADCS-Cert-Authority -IncludeManagementTools
```

Verify:

PowerShell

```
Get-WindowsFeature ADCS-Cert-Authority
```

You should see:

```
[X] AD CS Certification Authority
```

17. Configure the Enterprise subordinate CA

You can use Server Manager:

```
Server Manager
 → Manage
   → Add Roles and Features
```

If the role is already installed, proceed to:

```
Configure Active Directory Certificate Services
```

Select:

```
Certification Authority
```

CA type:

```
Enterprise CA
```

CA type/subtype:

```
Subordinate CA
```

Do **not** select Root CA.

18. Create the subordinate CA key

Choose:

```
Create a new private key
```

Cryptographic provider:

Use the standard Microsoft RSA provider available in your Server version.

Key length:

```
4096 bits
```

Hash:

```
SHA-256
```

CA name, for example:

```
Raccoon-Yellow-IssuingCA
```

Use your actual **classroom/team** name as required by your lab.

19. Generate the certificate request

The wizard should ask how to obtain the certificate from the parent CA.

Choose:

```
Send a certificate request to a parent CA later
```

or the equivalent option to save the request to a file.

Save it somewhere such as:

```
C:\PKI-Work\HQ-CA01.req
```

Do not send the private key.

The request contains the public-key request information. The private key remains on HQ-CA01.

20. Give the trainer the request

Give the trainer:

```
HQ-CA01.req
```

along with:

```
Classroom: Raccoon
Team:      Yellow
POD:       05
CA name:   Raccoon-Yellow-IssuingCA
```

The trainer signs the request on the classroom Root CA.

You should receive a subordinate CA certificate back.

For example:

```
HQ-CA01.cer
```

21. Install the issued subordinate CA certificate

On HQ-CA01:

Open:

```
certsrv.msc
```

Right-click the CA:

```
Certification Authority
 → All Tasks
 → Install CA Certificate...
```

Select the certificate returned by the trainer.

Then start the CA:

PowerShell

```
Start-Service CertSvc
```

Verify:

PowerShell

```
Get-Service CertSvc
```

Expected:

```
Status : Running
```

22. Verify the CA certificate

Run:

PowerShell

```
certutil -ca.cert C:\PKI-Work\HQ-CA01.cer
```

Then:

PowerShell

```
certutil -dump C:\PKI-Work\HQ-CA01.cer
```

Check:

```
Issuer = classroom Root CA
Subject = Raccoon-Yellow-IssuingCA
```

and:

```
Public Key Algorithm = RSA
Key size = 4096
Signature/hash = SHA-256
```

Most importantly, verify that the private key generated during the request is associated

with the installed certificate:

PowerShell

```
certutil -store My
```

You should see the CA certificate with:

```
Private key is NOT exportable
```

or equivalent indication that the private key is present.

23. Configure AIA/CDP

This is the part where we need to be especially careful.

Open:

```
certsrv.msc
```

Right-click:

```
Raccoon-Yellow-IssuingCA
 → Properties
 → Extensions
```

You need HTTP publication URLs based on:

```
http://pki.local/pki/
```

For example, if your actual subordinate CA certificate is:

```
Raccoon-Yellow-IssuingCA.crt
```

and its CRL is:

```
Raccoon-Yellow-IssuingCA.crl
```

then the URLs are:

```
http://pki.local/pki/Raccoon-Yellow-IssuingCA.crt
```

and:

```
http://pki.local/pki/Raccoon-Yellow-IssuingCA.crl
```

Use the actual filenames you configure/publish. Do not blindly copy these example names.

For AIA, ensure the certificate URL is included.

For CDP, ensure the CRL URL is included.

Retain the working local/AD publication locations required by the CA.

The HTTP URL is a **retrieval location**. It does not magically publish the files. We'll copy the actual files to `/opt/www/pki/` .

24. Restart CA and generate CRL

After changing CA Extensions:

PowerShell

```
Restart-Service CertSvc
```

Generate a new CRL:

cmd

```
certutil -crl
```

Find the generated CA certificate/CRL files using:

PowerShell

```
Get-ChildItem `
    "$env:windir\System32\CertSrv\CertEnroll" |
    Select-Object Name,Length,LastWriteTime
```

You should see files corresponding to:

```
Subordinate CA certificate
Subordinate CA CRL
```

25. Publish subordinate CA files to HQ-SRV

Copy the actual certificate and CRL files to:

```
/opt/www/pki/
```

For example:

Bash

```
sudo cp <CA-certificate> /opt/www/pki/
sudo cp <CA-crl> /opt/www/pki/
```

Set:

Bash

```
sudo chmod 644 /opt/www/pki/*
```

Do **not** copy:

```
.pfx
private key
.key
```

into this directory.

Verify:

Bash

```
ls -lh /opt/www/pki/
```

At this point you should have four public PKI objects:

```
RootCA.crt
RootCA.crl
<SubordinateCA>.crt
<SubordinateCA>.crl
```

26. Test all four URLs

From HQ-CA01:

PowerShell

```
curl.exe -I http://pki.local/pki/RootCA.crt
curl.exe -I http://pki.local/pki/RootCA.crl
curl.exe -I http://pki.local/pki/<SubordinateCA>.crt
curl.exe -I http://pki.local/pki/<SubordinateCA>.crl
```

Expected:

```
HTTP/1.1 200 OK
```

Then repeat from CP-APC or CP-DC03.

This is an important test because the CP path uses your WireGuard routing.

27. Publish the subordinate CA certificate in AD

Verify:

cmd

```
certutil -dspublish <SubordinateCA>.crt SubCA
```

Depending on the exact certificate and AD CS publication state, verify the CA is visible in AD:

PowerShell

```
Get-ADObject -LDAPFilter "(objectClass=certificationAuthority)" `
    -SearchBase "CN=Configuration,DC=purple,DC=raccoon,DC=lab" |
    Select-Object Name,DistinguishedName
```

Also verify NTAuth:

cmd

```
certutil -viewstore -enterprise NTAuth
```

The issuing CA should ultimately be trusted for the enterprise certificate workflows.

28. Before templates: verify the CA itself

Run:

cmd

```
certutil -ping
```

Then:

cmd

```
certutil -cainfo
```

And:

cmd

```
certutil -getreg CA\CACertPublicationURLs
certutil -getreg CA\CRLPublicationURLs
```

Also:

PowerShell

```
Get-Service CertSvc
```

You want:

```
Running
```

At this point **do not create the automatic enrollment GPO yet**.

First make sure:

```
Root CA trust             ✓
Subordinate CA            ✓
CA service                ✓
AIA URL                   ✓
CDP URL                   ✓
Root certificate HTTP     ✓
Root CRL HTTP             ✓
Subordinate certificate   ✓
Subordinate CRL           ✓
AD publication            ✓
```

29. Next phase: certificate templates

Once the above passes, we'll configure exactly two custom templates:

```
POD-Computer
POD-DomainController
```

with:

```
POD-Computer
 ├─ Domain Computers
 ├─ Read
 ├─ Enroll
 └─ Autoenroll

POD-DomainController
 ├─ Domain Controllers
 ├─ Read
 ├─ Enroll
 └─ Autoenroll
```

Then we'll create:

```
GPO-Certificate-Autoenrollment
```

and link it to:

```
Lab
Domain Controllers
```

Finally we'll test autoenrollment **individually on HQ-DC01, HQ-DC02, CP-DC03 and CP-DC04**, followed by the two FS servers, both APCs, and HQ-CA01.

Your immediate stopping point

For now, complete **Steps 1–16 above**, especially the Nginx publication test. The most useful checkpoint before touching AD CS is:

PowerShell

```
Resolve-DnsName pki.local
curl.exe http://pki.local/pki/test.txt
```

from **HQ-CA01**, and then the same URL from **CP-APC**.

Once `pki.local → 10.101.50.40` and HTTP works from both sites, proceed with the Root CA download/publication and subordinate CA request.

Friday 9:52 AM

PS C:\PKI-Work> certutil -f -dspublish RootCA.crl
A required CRL extension is missing
CertUtil: -dsPublish command FAILED: 0x80070490
(WIN32: 1168 ERROR_NOT_FOUND)
CertUtil: Element not found.

This error is specifically about the **CRL itself**, not AD permissions:

```
A required CRL extension is missing
CertUtil: -dsPublish command FAILED
0x80070490 ERROR_NOT_FOUND
```

`certutil -dspublish` expects the CRL to contain the information needed to determine **which CA's CRL publication point** it belongs to. Most commonly, the CRL is missing the **CRL Distribution Point (CDP) extension**.

1. Inspect the CRL first

On `HQ-CA01`, run:

PowerShell

```
certutil -dump C:\PKI-Work\RootCA.crl
```

Look for something similar to:

```
CRL Extensions
    CRL Number
    Authority Key Identifier
    Issuer
    ...
```

and especially whether there is a:

```
CRL Distribution Points
```

or CDP-related extension.

You can also use:

PowerShell

```
certutil -dump RootCA.crl | Select-String -Pattern "Distribution|CDP|CRL|Issuer"
```

2. Important distinction for your exercise

Your exercise explicitly says:

> If CRL publication cannot determine the AD location, obtain the correct DS CDP container/CN from the trainer; **do not invent**.

So **don't try random `-dspublish` parameters or manually create an AD container yet**.

The correct next step is to determine what is actually inside `RootCA.crl`.

Run:

PowerShell

```
certutil -dump RootCA.crl
```

and paste the **CRL Extensions** section here.

Also run:

PowerShell

```
certutil -dump RootCA.crt
```

and paste the **CDP/AIA-related extensions** from the Root CA certificate.

Then I can tell you whether:

1. the supplied `RootCA.crl` is missing the required extension;
2. the CRL is intended only for HTTP publication;
3. `certutil -dspublish` needs a specific DS CDP container/CN from your trainer; or
4. there is another issue with the Root CA files.

Do not regenerate or modify the classroom Root CA CRL yourself—the exercise expects the trainer-provided Root CA material.

PS C:\PKI-Work> certutil -dump C:\PKI-Work\RootCA.crl
X509 Certificate Revocation List:
Version: 2
Signature Algorithm:
Algorithm ObjectId: 1.2.840.113549.1.1.11 sha256RSA
Algorithm Parameters:
05 00
Issuer:
CN=CHQ-CA
Name Hash(sha1):
cb01d735fec94ecfdc53c634f64d5b5e16740c18

Show more

Yes — this output confirms the problem.

Your `RootCA.crl` is a valid **v2 CRL**, issued by:

```
Issuer: CN=CHQ-CA
```

but it contains **no CRL Distribution Point (CDP) extension**.

The four extensions present are:

- Authority Key Identifier
- CA Version
- CRL Number
- Next CRL Publish

There is no `2.5.29.31` (**CRL Distribution Points**) extension.

That explains exactly why:

PowerShell

```
certutil -f -dspublish RootCA.crl
```

returns:

```
A required CRL extension is missing
0x80070490 ERROR_NOT_FOUND
```

What this means for your lab

Do not modify or regenerate the classroom Root CA CRL. Your exercise specifically says that if AD publication cannot determine the DS CDP location, you should get the correct DS CDP container/CN from the trainer rather than inventing one.

The important distinction is:

Publication	Purpose	Your CRL
`certutil -dspublish RootCA.crt RootCA`	Publish Root CA certificate into AD	Should work
`certutil -dspublish RootCA.crl`	Publish Root CA CRL into AD using its CDP information	**Cannot determine location because CDP extension is absent**
Copy `RootCA.crl` to `/opt/www/pki/`	HTTP CRL retrieval	**Works**

So you can proceed with the **HTTP publication** independently.

First, verify the Root CA certificate's CDP

Run:

PowerShell

```
certutil -dump C:\PKI-Work\RootCA.crt
```

Find the section:

```
CRL Distribution Points
```

It should contain something like:

```
[1]CRL Distribution Point
     Distribution Point Name:
          Full Name:
               URL=http://...
```

Please paste that section.

Also, **don't retry** `-dspublish RootCA.crl` **yet**. If the Root CA certificate itself contains the classroom's CDP URL, that will tell us what the intended publication arrangement is. If there is no usable AD CDP information, we'll follow the exercise instruction and identify the DS CDP container from the trainer rather than guessing.



ChatGPT can make mistakes. Check important info.